UNIT 2

UNSUPERVISED LEARNING & ENSEMBLE LEARNING

Part 1: Unsupervised Learning Algorithms

Introduction to Unsupervised Learning

In supervised learning, our data has labels (e.g., "spam" or "not spam," "cat" or "dog"). The algorithm learns to map inputs to these known outputs. **Unsupervised Learning** is fundamentally different: we are given data without any labels. The goal is to find hidden patterns, intrinsic structures, or groupings within the data itself.

Key Idea: Let the data speak for itself.

Common Tasks:

1. **Clustering:** Grouping similar data points together.
2. **Association:** Finding rules that describe large portions of the data (e.g., "people who buy X also buy Y").
3. **Dimensionality Reduction:** Reducing the number of variables while preserving the essential information.

---

1. Clustering: K-Means Clustering

K-Means is one of the most popular and simple clustering algorithms. Its goal is to partition n data points into k distinct, non-overlapping clusters.

The Intuition

Imagine you have a bag of mixed-color marbles (red, blue, green). Your task is to sort them into three bowls (k=3). You would naturally pick a marble, decide which color group it belongs to, and place it in the corresponding bowl. K-Means automates this process.

The Algorithm Step-by-Step

Step 1: Initialization
Choose the number of clusters, k. Randomly select k data points from the dataset to serve as the initial **centroids** (the centers of the clusters).

Step 2: Assignment Step
For each data point in the dataset:

- Calculate the distance (usually Euclidean distance) to each of the k centroids.
- Assign the data point to the cluster whose centroid is the closest.

Step 3: Update Step
For each cluster, recalculate the centroid. The new centroid is the mean (average) of all data points currently assigned to that cluster. This is why it's called "K-Means."

Step 4: Iteration
Repeat Steps 2 and 3 until:

- The centroids no longer change significantly (they have converged).
- The assignments of data points to clusters no longer change.
- A predetermined number of iterations is reached.

Example: Customer Segmentation

Dataset: Customer data with two features: Annual Income and Spending Score.
Goal: Group customers into segments for targeted marketing.

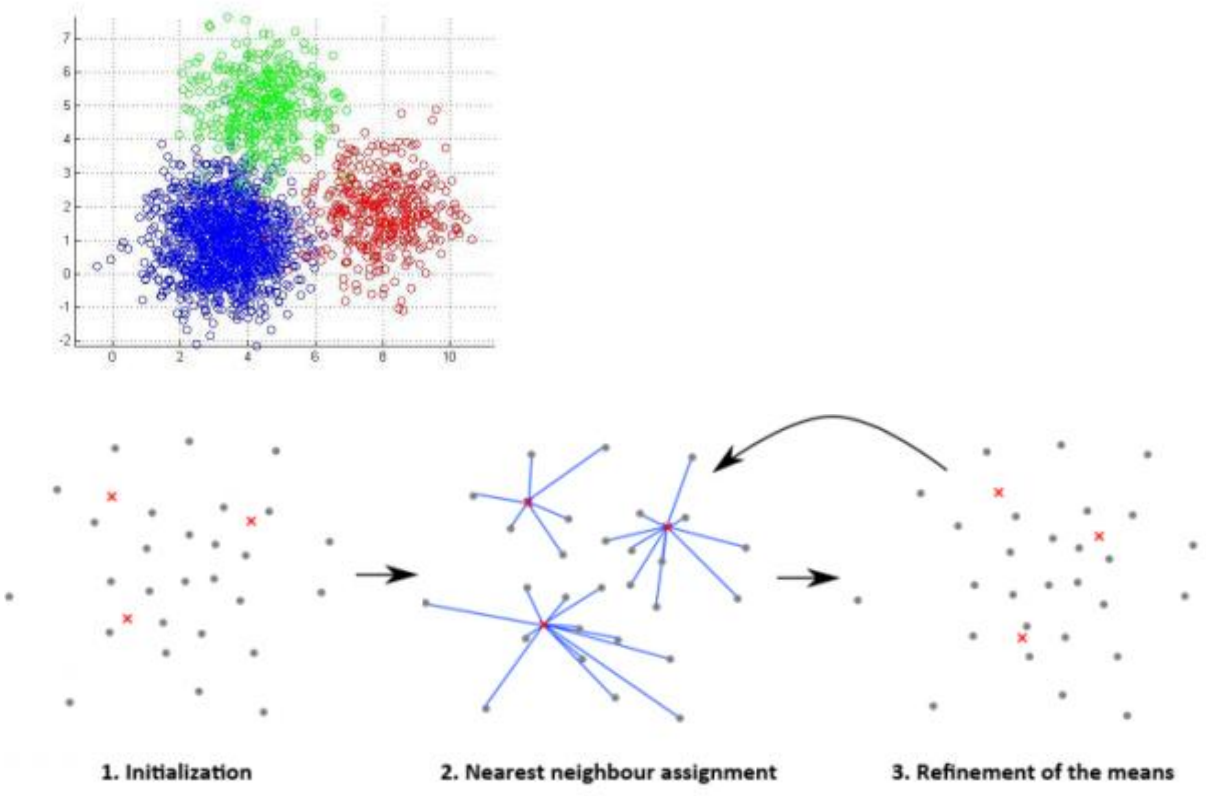
1. **Choose k=3** (we want 3 segments: Budget, Average, Premium).
2. **Initialization:** Randomly pick 3 customers as initial centroids.
3. **Assignment:** For every customer, calculate the distance to these 3 centroids. Assign them to the nearest one.
4. **Update:** For the group of customers assigned to "Centroid 1," calculate the mean Annual Income and mean Spending Score. This new mean point becomes the new "Centroid 1." Repeat for the other clusters.
5. **Repeat** until the clusters stabilize.

You might end up with:

- **Cluster 1 (Budget):** Low Income, Low Spending
- **Cluster 2 (Premium):** High Income, High Spending
- **Cluster 3 (Careful Spenders):** High Income, Low Spending

Challenges and Considerations

- **Choosing k:** This is the biggest challenge. The algorithm doesn't know how many clusters exist. Common methods to choose k include the **Elbow Method** (plotting the within-cluster variance against k and looking for an "elbow" bend) and domain knowledge.
- **Sensitivity to Initialization:** Random initial centroids can lead to different final clusters. Solution: Run the algorithm multiple times and pick the best result.
- **Sensitivity to Outliers:** Outliers can skew the mean calculation.



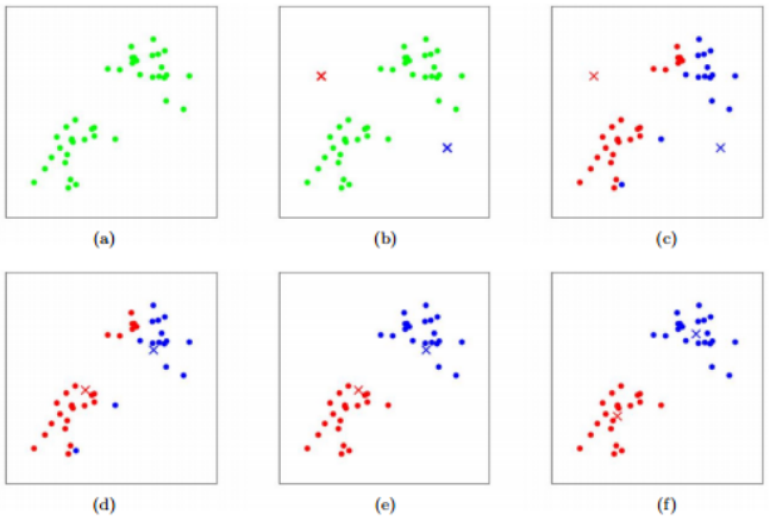


Figure 1: K-means algorithm. Training examples are shown as dots, and cluster centroids are shown as crosses. (a) Original dataset. (b) Random initial cluster centroids. (c-f) Illustration of running two iterations of k-means. In each iteration, we assign each training example to the closest cluster centroid (shown by "painting" the training examples the same color as the cluster centroid to which is assigned); then we move each cluster centroid to the mean of the points assigned to it. Images courtesy of Michael Jordan.

The Algorithm

In the clustering problem, we are given a training set ${x^{(1)}, ... , x^{(m)}}$, and want to group the data into a few cohesive "clusters." Here, we are given feature vectors for each data point $x^{(i)} \in \mathbb{R}^n$ as usual; but no labels $y^{(i)}$ (making this an unsupervised learning problem). Our goal is to predict k centroids **and** a label $c^{(i)}$ for each datapoint. The k-means clustering algorithm is as follows:

1. Initialize **cluster centroids** $\mu_1, \mu_2, \ldots, \mu_k \in \mathbb{R}^n$ randomly.
2. Repeat until convergence: {

   For every i, set
   $$c^{(i)} := \arg\min_j ||x^{(i)} - \mu_j||^2.$$

   For each j, set
   $$\mu_j := \frac{\sum_{i=1}^{m} 1\{c^{(i)} = j\} x^{(i)}}{\sum_{i=1}^{m} 1\{c^{(i)} = j\}}.$$

   }

---

2. Hierarchical Clustering

Unlike K-Means, which requires pre-specifying k, Hierarchical Clustering creates a tree-like structure of clusters (a **dendrogram**) that allows you to view clusters at different levels of granularity.

Two Main Approaches:

a) Agglomerative (Bottom-Up)
This is the most common approach.

1. **Start:** Treat each data point as its own singleton cluster. So, if you have n points, you have n clusters.
2. **Merge:** Repeatedly merge the two clusters that are the most similar (closest) until all points are in a single cluster.

b) Divisive (Top-Down)

1. **Start:** All data points belong to one single cluster.
2. **Split:** Recursively split the cluster into two least similar clusters until each cluster contains a single point.

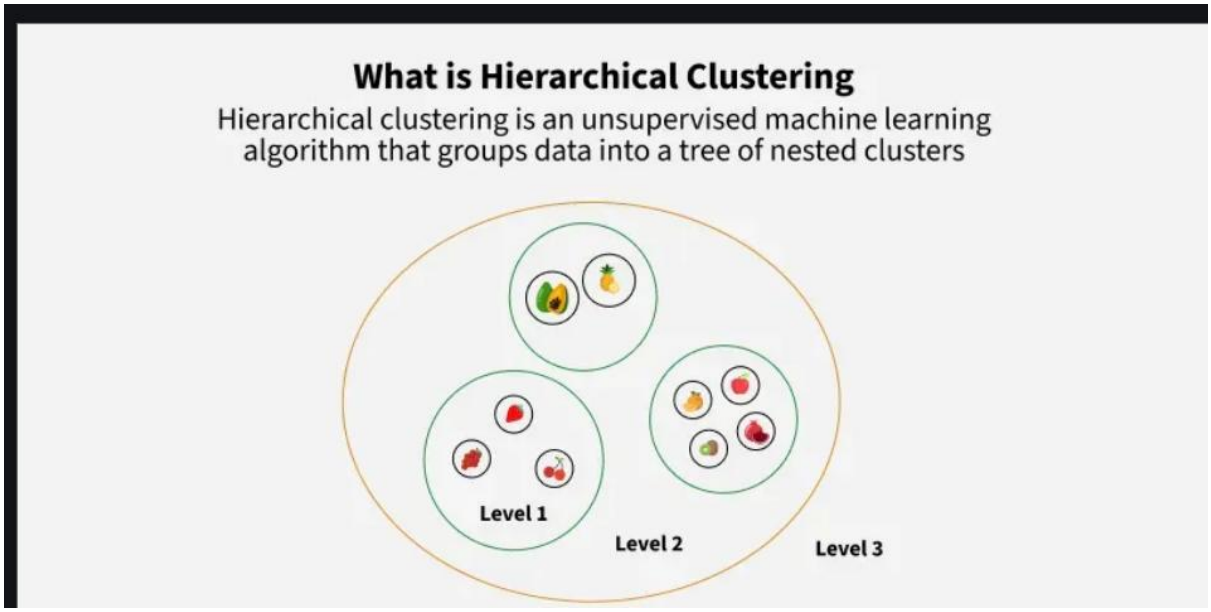
Key Concepts: The Dendrogram and Linkage

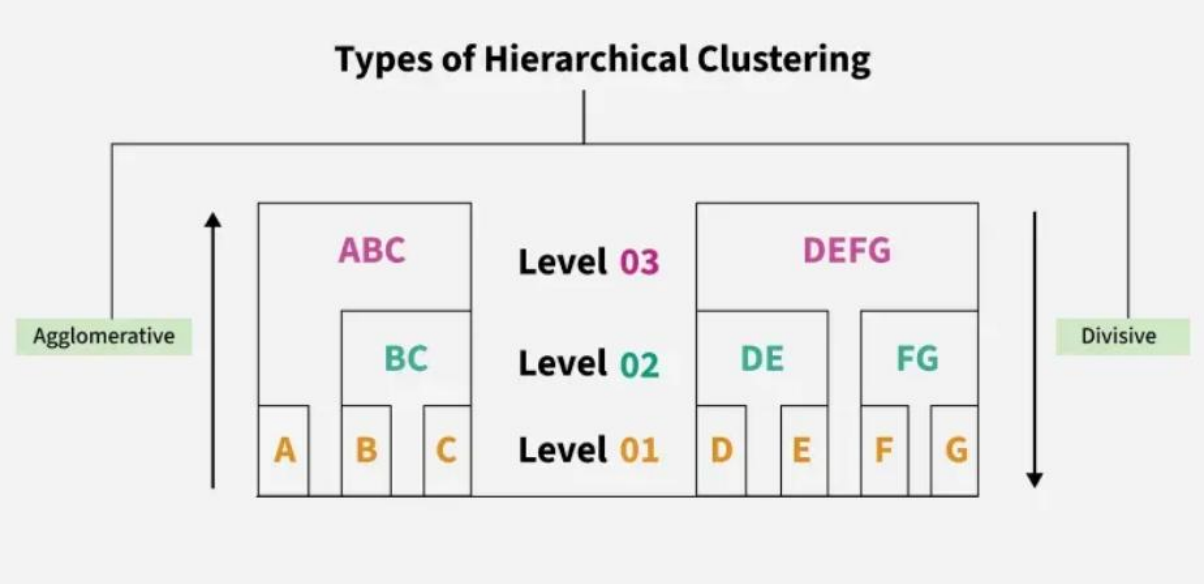
- **Dendrogram:** A tree diagram that records the sequence of merges (or splits) and the distance at which each merge occurred. The height of the merge points represents the distance between the clusters. **You can decide the number of clusters by "cutting" the dendrogram at a certain height.**

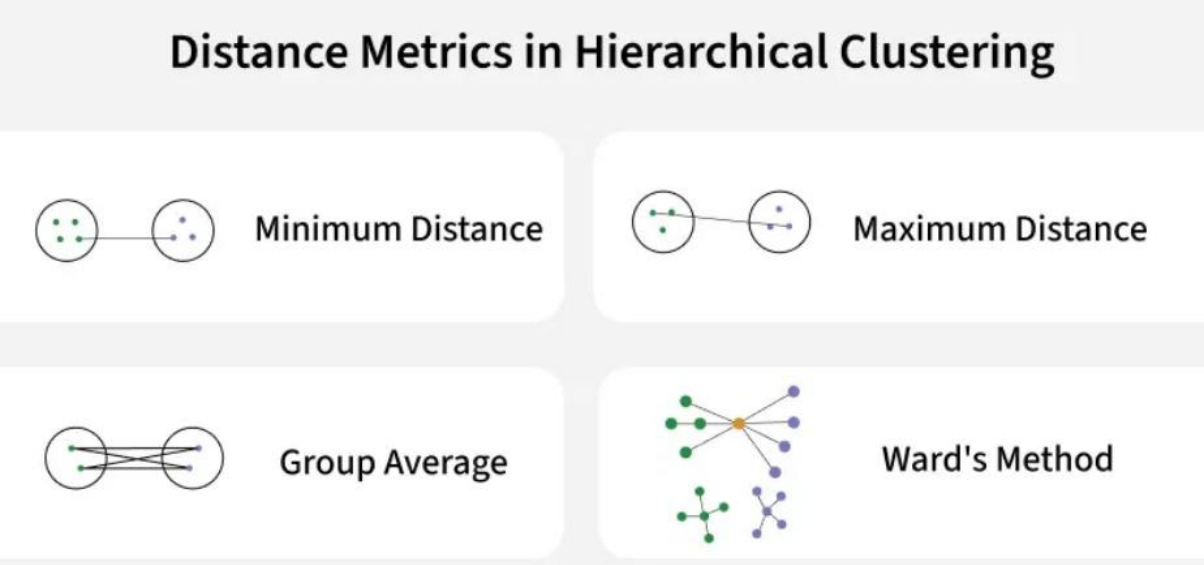
- **Linkage Criteria:** How do we define "closest" between two clusters that contain multiple points?
    - **Single Linkage:** Distance between the closest points of the two clusters. Can lead to long, "chain-like" clusters.
    - **Complete Linkage:** Distance between the farthest points of the two clusters. Tends to find compact, spherical clusters.
    - **Average Linkage:** Average distance between all pairs of points in the two clusters. A good compromise.
    - **Ward's Method:** Merges clusters that lead to the smallest increase in total within-cluster variance. Similar to K-Means' objective.

Example: Animal Clustering

Imagine clustering animals based on features like size, diet, and habitat.
The dendrogram might first group a "Cat" and a "Lion" together (both felines). Then it might group "Dog" and "Wolf" together (both canines). At a higher level, the feline group and canine group might merge into a "Carnivorous Land Mammals" cluster. This hierarchical structure is very informative.







---

3. Probabilistic Clustering & Gaussian Mixture Models (GMM):

K-Means assigns each point to a single cluster (hard assignment). What if a point could belong to multiple clusters with certain probabilities? This is the idea behind probabilistic clustering.

Gaussian Mixture Model (GMM) is the most famous example. It assumes that all data points are generated from a mixture of a finite number of Gaussian (Normal) distributions with unknown parameters.

The Intuition

Imagine the data points are heights of people from two different countries. The height distribution in each country is roughly a bell curve (Gaussian), but the means and variances might be different. A GMM would try to learn the parameters (mean, variance) of each country's bell curve and the mixing proportion (how many people are from each country). A person's height then gives a probability that they are from Country A or Country B.

The Mathematics

A GMM is defined by:

- **Number of components (k):** The number of clusters/Gaussian distributions.
- **Mixing coefficients (π):** The probability that a randomly chosen data point was generated by a particular component. ($\sum\pi = 1$).
- **Component parameters (μ, σ):** The mean and covariance (multidimensional variance) of each Gaussian distribution.

The Expectation-Maximization (EM) Algorithm for GMM

The EM algorithm is used to find the parameters of the GMM. It's an iterative process with two steps:

1. **Expectation (E-Step):** Given the current model parameters, calculate the probability that each data point belongs to each cluster. This is the "soft assignment" or **responsibility**.
2. **Maximization (M-Step):** Given these probabilities, update the model parameters (means, covariances, mixing coefficients) to maximize the likelihood of the data. This is like a "weighted" K-Means update.

The algorithm alternates between these two steps until convergence.

GMM vs. K-Means

- **K-Means:** Hard assignment, clusters are spherical. Fast and simple.
- **GMM:** Soft assignment, can identify clusters of different shapes and sizes (using covariance). More flexible but slower.

---

4. Association Rule Mining: Apriori Algorithm

This technique is used to discover interesting relationships (association rules) in large datasets, famously used for **Market Basket Analysis**.

Key Terminology

- **Itemset:** A collection of one or more items. E.g., {Milk, Bread}.
- **Support:** The frequency of an itemset. How often does {Milk, Bread} appear together in all transactions?

    - Support(Milk, Bread) = (Number of transactions containing Milk and Bread) / (Total number of transactions)
- **Confidence:** How often the rule is found to be true.
    - Confidence(Milk -> Bread) = Support(Milk, Bread) / Support(Milk)
    - "What is the probability that a customer who buys Milk also buys Bread?"
- **Lift:** Measures the strength of a rule over the random co-occurrence of Antecedent and Consequent.
    - Lift(Milk -> Bread) = Support(Milk, Bread) / (Support(Milk) * Support(Bread))
    - **Lift > 1:** The rule is useful (Milk and Bread are positively correlated).
    - **Lift = 1:** The rule is random.
    - **Lift < 1:** The rule is misleading (the items are substitutes).

The Apriori Algorithm: "A Priori" means "prior knowledge."

The algorithm uses a key principle: **If an itemset is frequent, then all its subsets must also be frequent.** Conversely, if an itemset is infrequent, all its supersets will be infrequent.

Step-by-Step:

1. **Set a minimum support threshold.**
2. **Find all frequent 1-itemsets:** Items (e.g., Milk, Bread, Diaper) that have support above the threshold.
3. **Join Step:** Use frequent 1-itemsets to generate candidate 2-itemsets (e.g., {Milk, Bread}, {Milk, Diaper}).
4. **Prune Step:** Prune candidates that have an infrequent subset (using the Apriori principle).
5. **Check support** for the remaining candidates to find frequent 2-itemsets.
6. **Repeat** the Join and Prune steps to find frequent 3-itemsets, 4-itemsets, etc., until no more frequent itemsets can be found.
7. **Generate Rules:** From the frequent itemsets, generate association rules that meet a minimum confidence threshold.

Example: Supermarket Transactions

Transaction Data:

1. {Milk, Bread}
2. {Milk, Diaper, Beer, Eggs}
3. {Bread, Diaper, Beer, Cola}

4. {Bread, Milk, Diaper, Beer}
5. {Bread, Milk, Diaper, Cola}

A Rule we might find: {Diaper} -> {Beer}

- **Support({Diaper, Beer})** = 3/5 = 0.6
- **Confidence** = Support({Diaper, Beer}) / Support({Diaper}) = (3/5) / (4/5) = 0.75

This means 75% of the time when a diaper is bought, beer is also bought.

Part 2: Ensemble Learning

Ensemble Learning combines multiple machine learning models (often called "weak learners") to produce a single, more powerful, and robust model. The idea is that a group of weak models, when combined, can outperform any single one of them. It's the machine learning equivalent of "the wisdom of the crowd."

1. Bagging (Bootstrap Aggregating)

Core Idea: Reduce variance and prevent overfitting by training multiple models in parallel on different subsets of the training data and then averaging their predictions (for regression) or taking a majority vote (for classification).

How it Works:

1. **Bootstrap Sampling:** Create multiple new training sets (called bootstrap samples) by randomly sampling from the original training data *with replacement*. This means some data points may appear multiple times in a sample, while others may be left out (these are "out-of-bag" samples).
2. **Parallel Training:** Train a base model (e.g., a Decision Tree) on each of these bootstrap samples. These models are trained independently.
3. **Aggregation:** For a new data point, get the prediction from every trained model and combine them.

Random Forest: The Prime Example of Bagging

A Random Forest is an ensemble of Decision Trees, trained via the bagging method, with one crucial extra tweak: **Feature Randomness**.

- When building each tree, at each split, the algorithm is not allowed to choose from all features. Instead, it must choose from a random subset of features.
- This de-correlates the trees even further. If one or a few features are very strong predictors, they would dominate all trees without this randomness. By forcing each tree to consider only a subset, we ensure a diverse set of trees.

- The final prediction is the majority vote (classification) or average (regression) of all trees.

Advantages: Highly accurate, resistant to overfitting, can handle large datasets with higher dimensionality.

---

2. Boosting

Core Idea: Reduce bias by training multiple models sequentially, where each new model focuses on the mistakes made by the previous ones. Boosting creates a *strong learner* from a collection of *weak learners*.

How it Works:

1. **Sequential Training:** Train a weak model (e.g., a shallow Decision Tree, called a "stump") on the entire dataset.
2. **Weighting Mistakes:** The model will make some errors. Increase the weight of the data points that were misclassified.
3. **Next Model:** Train the next model on the re-weighted data. This new model will pay more attention to the previously misclassified points.
4. **Iteration:** Repeat this process for a number of rounds.
5. **Combination:** The final model is a weighted sum of all the weak models, where models that performed better have a higher weight.

AdaBoost (Adaptive Boosting)

One of the first successful boosting algorithms.

- It fits a sequence of weak learners (e.g., stumps) on repeatedly modified versions of the data.
- The predictions are combined through a weighted majority vote.

Gradient Boosting Machines (GBM)

A more generalized and powerful framework. Instead of adjusting weights, Gradient Boosting:

- Trains each new model to predict the **residual errors** (the difference between the current prediction and the true value) of the previous model.
- It uses gradient descent to minimize a loss function (like Mean Squared Error).
- **XGBoost (Extreme Gradient Boosting)** is a highly optimized and popular implementation of GBM known for its speed and performance.

---

3. Stacking (Stacked Generalization)

Core Idea: Instead of using simple functions (like averaging) to combine predictions, use a machine learning model to learn how to best combine the predictions of the base models.

How it Works:

1. **Level-0 Models (Base Models):** Train several different models on the training data (e.g., a K-NN, an SVM, and a Decision Tree). These are diverse models.
2. **Create a New Dataset:** Use these base models to make predictions on a validation set (or via cross-validation). The predictions from these models become the new features (meta-features) for the next level.
3. **Level-1 Model (Meta-Model):** Train a new model (the meta-model, e.g., a Logistic Regression) on this new dataset. The inputs are the predictions from the base models, and the output is the true target value.
4. **Final Prediction:** To predict a new instance, first get predictions from all base models, then feed these predictions as input to the meta-model to get the final prediction.

Analogy: Think of base models as a panel of expert witnesses (e.g., a doctor, an engineer, a lawyer). The meta-model is the jury, who listens to all the expert testimonies (predictions) and learns how to weigh them to make the final, most informed decision.

Summary Table: Ensemble Methods

Feature	Bagging (e.g., Random Forest)	Boosting (e.g., AdaBoost, GBM)	Stacking
Goal	Reduce Variance	Reduce Bias	Optimize Combination
Training	Parallel	Sequential	Two-Stage (Base then Meta)
Focus	Independent models on data samples	Correcting previous model's errors	Learning a combiner model
Base Models	Often the same type (Homogeneous)	Often the same type (Homogeneous)	Should be different (Heterogeneous)
Overfitting	Resistant to overfitting	Can overfit if too many models are used	Can overfit if not careful with meta-data